Lecture 5
Vim, Permissions

Comp 206 – Introduction to Software Systems – Fall 2026

Mathieu Bérubé-Vallières

McGill University

September 15, 2026

External announcements

Bérubé-Vallières © 2026
See last slide for complete credits

PharmaHacks

- PharmaHacks is recruiting for our 2025-2026 team! We are a biotech and AI hackathon. This is a great way to apply your machine learning skills! To apply, please fill out this form:
- VP position: https://forms.gle/2SSSrGc4TmHgUAvu5
- Committee position: https://forms.gle/itFHLbmoNwNh9nwd6
- Our application deadline is Friday, September 27th, 2026



TechFair

- TechFair is a career fair that runs twice a year for engineering and computer science students, organized by a committee of student volunteers, and by the Engineering Career Centre (ECC). As we are fully volunteer-run, it would mean a great deal to us if you could share the following advertisement post on your My Courses page for your students to see.
- The fair is scheduled as follows:
    - **Day 1: September 30th (Wednesday) - 10 am to 3 pm**
    - **Day 2: October 1st (Thursday) - 10 am to 3 pm**
- As our event continues to grow, we are always in need of dedicated volunteers to ensure its success. Volunteering comes with several perks, including:
    - Food (Pizza, samosas and snacks) **we also have gluten-free & vegetarian options**
    - More access to companies
    - Line Skip
    - Free merch
    - Volunteer Certificate
- And beyond these, it's a super simple as well as fun way to get involved and meet new people from different programs and years, so why not?
- If you are interested in contributing as a volunteer, please sign up using the following link:
    - [Volunteer Signup Form]
- Thank you for considering this opportunity, and we look forward to your support in making this semester TechFair a resounding success.



Comp 206 announcements

Bérubé-Vallières © 2026
See last slide for complete credits

TA activities

- Tutorials
- Mentorship/OH appointments
- Study sessions

Bérubé-Vallières © 2026
See last slide for complete credits

Final exam attendance

- Exam will be held by Exam Office instead of in-department
- ⇒ Final exam is still optional (can leave exam blank if satisfied with your midterm grades), but attendance is mandatory

Bérubé-Vallières © 2026
See last slide for complete credits

Vim

Bérubé-Vallières © 2026
See last slide for complete credits

Vim

- Powerful text editor designed to efficiently write text from a terminal
- Preinstalled on most Linux distributions
- Modal editor
  - Keys have different functions depending on current mode
  - (compare to editors like Notepad where typing always inserts text)

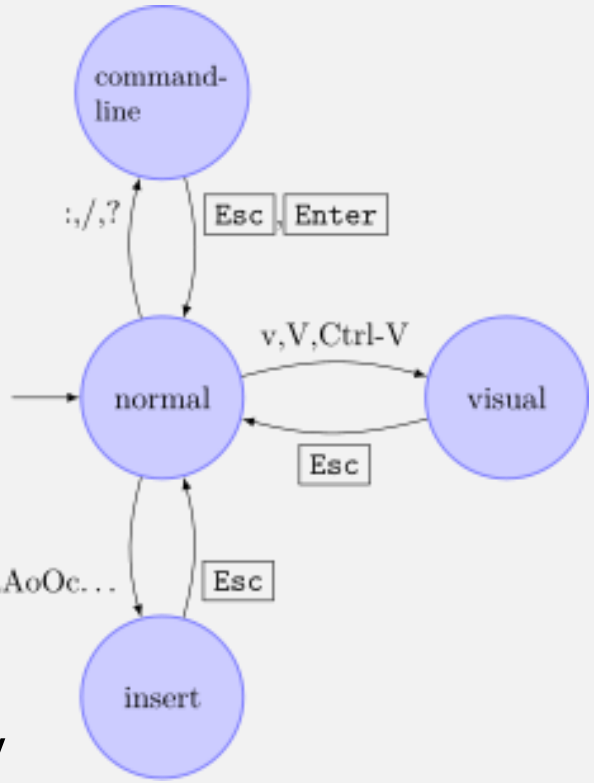
Bérubé-Vallières © 2026
See last slide for complete credits

Vim modes

- Instead of a menu system, Vim uses *modes*

The most important modes are:

- Insert mode:
  - To edit your text
  - Can press any keyboard characters
- Normal mode (also known as command mode):
  - Enter editor commands
    - Move around in the file, change modes, copy, undo, etc.
- Command-line mode:
  - Save, load, quit, search, etc.
- Visual mode:
  - Like normal modes, but movement keys highlight lines to apply editor commands on all of them



Bérubé-Vallières © 2026
See last slide for complete credits

Important commands (1/2)

To get help for a command
:help command

- Insert mode:
  - To get in insert mode, any of the following: i, I, a, A, o, O
    - i – insert text before the cursor | I – insert at start of line before first non-blank
    - a – append text after the cursor | A – append text at the end of the line
    - o – begin a new line below the cursor and insert text | O – same but above cursor
- Normal mode:
  - To get to to normal mode: Esc key
  - To move:
    - h, j, k, l to move left, down, up, right
      - (arrows also work but not recommended)
    - gg – go to first line | G – go to last line
      - (many others like w to go to next word, [(to go to next unmatched (, etc.)
  - To delete: dd, D, x, r
    - dd – delete a line | D – delete the rest of the line
    - x – delete a character | r – replace a character
  - To copy (yank) and paste a line:
    - yy or Y (equivalent) – yank | p - paste after cursor | P – paste before cursor
  - To undo and redo:
    - u (undo) and ctrl-r (redo)

Bérubé-Vallières © 2026
See last slide for complete credits

Important commands (2/2)

To get help for a command
:help command

- Command-line mode:
  - w, q, wq, q!, line number, e filename
    - :w – save file | :q – quit current window | :wq save and quit | :q! quit and discard changes
    - :number – go to a specific line number (For example, :14)
    - :e filename – Edit a file
  - To search: / (forward), ? (backwards)
    - Once you press enter, press n and N to get next/previous result

Bérubé-Vallières © 2026
See last slide for complete credits

Demo

Bérubé-Vallières © 2026
See last slide for complete credits

More on Vim

- Commands can be preceded by a number to repeat them a number of times, for example `5dd` would delete the next 5 lines. (In the help documentation this is described as `[count]`)
- You can learn about basic vim commands by typing vimtutor in your Unix command-line. This is a program that comes with every vim installation
- There is a lot more you can do with Vim! For example, you can execute shell commands directly from the command-line mode (for example, compile and run the program that you're currently editing without leaving Vim, etc.), you can save text in different registers for later use (including the outputs of shell commands!), you can use macros to repeat common sequences, etc.
  - Only have to know the important commands from previous slides, but recommend learning more!

Bérubé-Vallières © 2026
See last slide for complete credits

More on Vim

- Commands can be preceded by a number to repeat them a number of times, for example `5dd` would delete the next 5 lines. (In the help documentation this is described as `[count]`)
- You can learn about basic vim commands by typing vimtutor in your Unix command-line. This is a program that comes with every vim installation
- There is a lot more you can do with Vim! For example, you can execute shell commands directly from the command-line mode (for example, compile and run the program that you're currently editing without leaving Vim, etc.), you can save text in different registers for later use (including the outputs of shell commands!), you can use macros to repeat common sequences, etc.
  - Only have to know the important commands from previous slides, but recommend learning more!

Bérubé-Vallières © 2026
See last slide for complete credits

Fun fact: Vim name

- QED: *q*uick *ed*itor
- ed: derived from QED: the original Unix text *ed*itor
- em: improvement of ed (stands for *e*d for *m*ortals)
- en: improvement of em
- ex: improvement of en, stands for *ex*tended
- vi: the *vi*sual mode of ex (could use vi to start ex in visual mode): eventually, became more common to use vi directly
- STEVIE: *ST E*ditor for *VI E*nthusiast
- Vim: *vi* *im*proved, based on STEVIE (originally *vi* *im*itation)

Bérubé-Vallières © 2026
See last slide for complete credits

Permissions

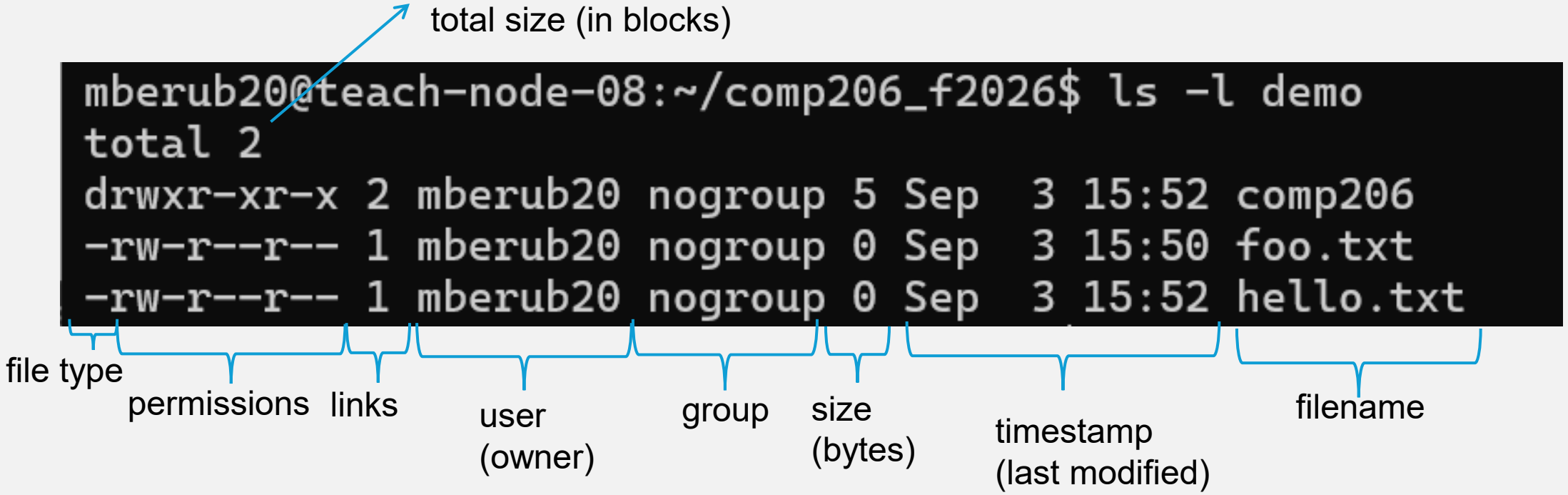
Bérubé-Vallières © 2026
See last slide for complete credits

Permissions

- All files are **owned** by a user and a group.
  - Usually, this owner is the user that created the file.
- Permissions on files exists at three level (classes):
  - **user**, **group** and **others**.
- Three types of rights can be given:
  - **read**, **write** and **execute**.
- Any combination of these rights must be given to these three levels.

Bérubé-Vallières © 2026
See last slide for complete credits

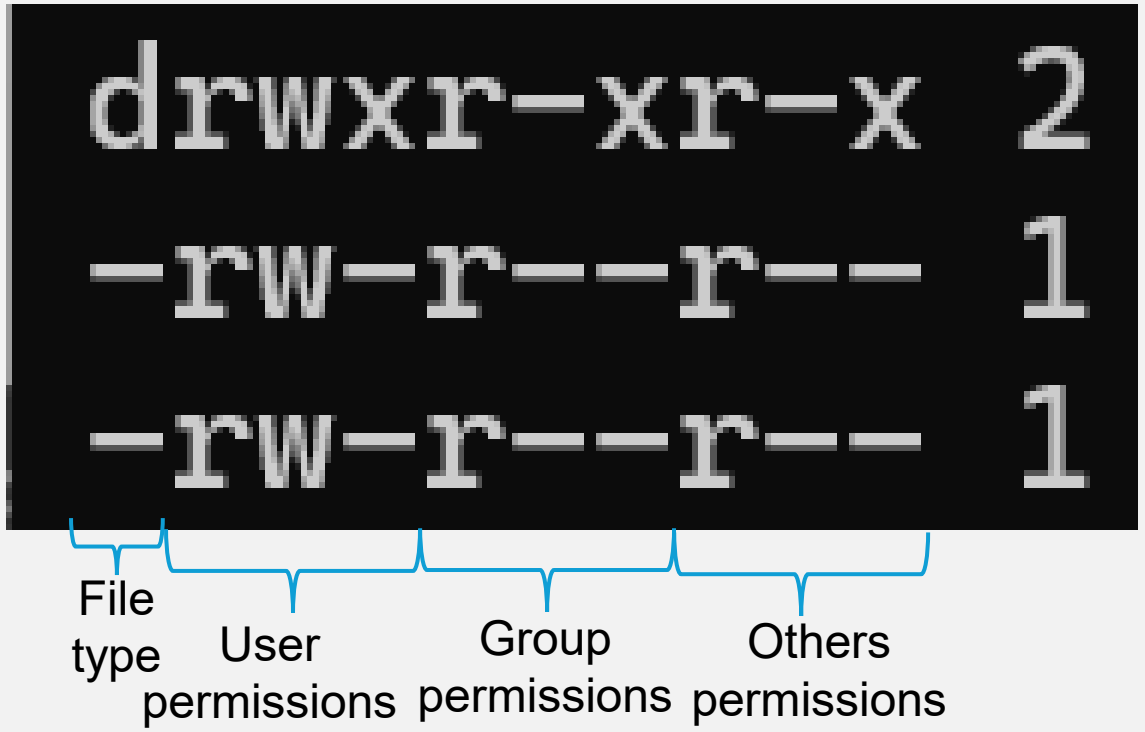
Recall: `ls -l`



Bérubé-Vallières © 2026
See last slide for complete credits

Symbolic notation

- The symbolic notation for permissions is displayed as a string of 9 characters (10 if including file type)
  - (0th : indicates if the file is a directory.)
  - 1st : indicates if the user that owns the file has read access to the file.
  - 2nd : indicates if the user that owns the file has write access to the file.
  - 3rd : indicates if the user that owns the file has execute access to the file.
  - 4th , 5th , 6th : indicates if the group owner has read, write or execute.
  - 7th , 8th, 9th : indicates if all other users have read, write or execute.



Order of precedence for permissions

- The order of precedence for permissions is
  - User
  - Group
  - Others
- If a user is in multiple classes, the highest one applies

Bérubé-Vallières © 2026
See last slide for complete credits

Order of precedence for permissions

- The order of precedence for permissions is
  - User
  - Group
  - Others
- If a user is in multiple classes, the highest one applies
- Can create surprising situations:
  Given permissions “------rwx” of a file I own, can I read the file?

Bérubé-Vallières © 2026
See last slide for complete credits

Order of precedence for permissions

- The order of precedence for permissions is
  - User
  - Group
  - Others
- If a user is in multiple classes, the highest one applies
- Can create surprising situations:
  Given permissions “------rwx” of a file I own, can I read the file?
  - I cannot read the file
  - People in the group cannot read the file
  - Others will be able to read the file

Bérubé-Vallières © 2026
See last slide for complete credits

`chmod` command

- **Ch**ange **mod**e: Change file permissions
  - Can be used by owner or superuser (root)
- Who:
  - u : The user who owns the file (this means “you.”)
  - g : The group the file belongs to.
  - o : The other users
  - a : all of the above (an abbreviation for ugo)
- Permission
  - r : Permission to read the file.
  - w : Permission to write (or delete) the file.
  - x : Permission to execute the file (for directories, this is needed to enter them)
- Changes to
  - = : become
  - + : add
  - - : remove

Bérubé-Vallières © 2026
See last slide for complete credits

`chmod` command examples

- The syntax of the command is as follows:
  - `chmod who=permission files`

Here are a few examples of the chmod command:

- Give read permission to group
  - `chmod g+r file.txt`
- Give write/execute permission to you (user)
  - `chmod u+wx file2.txt`
- Remove all permissions from others
  - `chmod o= file3.txt`
- Give read/write permission only to user and group
  - `chmod ug=rw file4.* file2.txt`
- Give read only to user and execute only to group
  - `chmod u=r,g=x file4.* file2.txt` (no space after the comma)

Bérubé-Vallières © 2026
See last slide for complete credits

Binary representation of permissions

- Given a 9 character permission string:

rwx --- rw-

- We can equivalent represent it in binary

111 000 110

- where 1 means “on” and 0 means “off”

Example: What does 010 111 001 represent?

Bérubé-Vallières © 2026
See last slide for complete credits

Binary representation of permissions

- Given a 9 character permission string:

rwx --- rw-

- We can equivalent represent it in binary

111 000 110

- where 1 means “on” and 0 means “off”

Example: What does 010 111 001 represent?

- Answer: -w- rwx --x

Bérubé-Vallières © 2026
See last slide for complete credits

Octal representation of permissions

- Given the binary representation of permissions, we can convert it to *octal* (base 8) by converting each group of 3 bits to its corresponding digit as follows:

Binary	Octal
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Octal representation of permissions

- For example, given

111 000 110

- The octal representation is

706

Binary	Octal
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Octal representation of permissions

- For example, given

111 000 110

- The octal representation is

706

Binary	Octal	Symbolic
000	0	`---`
001	1	`--x`
010	2	`-w-`
011	3	`-wx`
100	4	`r--`
101	5	`r-x`
110	6	`rw-`
111	7	`rwx`

`chmod` and octal representation

- `chmod` supports octal representation
  - Often used when want to set up all permissions at the same time
- Ex:
  - chmod 400 filename
    - Give user permission to read and no permissions to group and others
  - chmod 644 filename
    - Give user permission to read and write and only give permission to read to group and others
  - chmod 700 filename
    - Give user all permissions and no permissions to group and others

Binary	Octal	Symbolic
000	0	`---`
001	1	`--x`
010	2	`-w-`
011	3	`-wx`
100	4	`r--`
101	5	`r-x`
110	6	`rw-`
111	7	`rwx`

Bérubé-Vallières © 2026
See last slide for complete credits

Additional credits

- Some content from this lecture is based on:
  - Past Comp 206 slides from Prof. Vybihal

Bérubé-Vallières © 2026
See last slide for complete credits